

EXPERIMENT REPORT · DEEP LEARNING

Hierarchical Expert Networks

A comparative study of flat vs. hierarchical CNN classifiers across five scripts, with 5-way router for multi-script handwriting recognition.

- 1. Overview
- 2. Dataset
- 3. Model Comparison
- 4. Architectures
- 5. Improvement Attempts
- 6. Error Analysis
- 7. Challenges & Solutions
- 8. Key Findings
- 9. Conclusion

1 Overview

This experiment trains and evaluates **three** neural network classifiers on a **54-class** handwriting dataset spanning **five scripts**: Arabic digits (0–9), Latin uppercase letters (A–J, excl. I), Chinese numeral characters (一—億, 15 classes), Urdu characters (10 selected), and **Kannada digits** (official **Kannada-MNIST**, 10 classes). Two architecture families are explored—**CNN** and **MLP**—in flat baselines, plus a hierarchical tree-structured CNN variant. The hierarchical CNN uses a **5-way router** to cleanly separate all five scripts.

*Core question: can a tree-structured "router + expert" architecture match a monolithic classifier in accuracy, while reducing active parameters at inference, improving modularity, and **scaling seamlessly to new scripts**?*

2 Dataset

Dataset	Classes	Train	Test	Original Size	Unified Size	Label Range
MNIST Digits	10	60,000	10,000	28×28	32×32	0–9
EMNIST Letters (A–J, excl. I)	9	48,488	8,113	28×28 (rotated)	32×32	10–18
Chinese MNIST	15	12,000	3,000	64×64	32×32	19–33
UHaT Urdu Characters	10	7,540	1,353	28×28	32×32	34–43
Kannada-MNIST	10	60,000	10,000	28×28	32×32	44–53
Combined	54	188,028	32,466	—	32×32 gray	0–53

Notes: EMNIST raw images are stored rotated 90° CCW — corrected via transpose + flip. Chinese MNIST has no official split; stratified 80/20 split applied (seed=42). The Urdu dataset is [UHaT](#) (28×28 .npz); 10 visually distinctive characters selected from 40 classes and remapped to labels 34–43. Kannada uses the official Kannada-MNIST release (60,000 train / 10,000 test, 28×28 grayscale), resized to 32×32 and remapped to labels 44–53. All images normalized to mean=0.5, std=0.5.

3 Model Comparison

Model	Type		Total Params	Dormant @ Inference	Test Accuracy	VRAM	Infer. Time
Flat CNN	Flat	CNN	631,318	0 (0%)	99.08%	108 MB	0.63 ms
Hierarchical CNN (5-way)	Hier	CNN	632,603	~126k (20%)	98.98%	113 MB	0.93 ms
Flat MLP	Flat	MLP	671,542	0 (0%)	92.48%	30 MB	0.39 ms

Hierarchical CNN dormant params estimated as the mean inactive expert head size at inference (4 of 5 expert heads are idle per sample: $\sim 4 \times 31.5k = \sim 126k$ / 20%). All models trained on RTX 3080 (PyTorch 2.5.1 + CUDA). Flat CNN/MLP: 20 epochs AdamW + cosine LR. Hierarchical CNN: warm-up (3 ep) coarse router, then 22 joint epochs with masked CE on the official Kannada-MNIST split.

4 Architectures

Flat CNN · 631,318 params · 3-block backbone, single output head (54 classes)

Input → [Conv] → [Conv] → [Conv] → Flatten → FC(256) → Linear(256→54)

▲ All parameters always active, no routing overhead. Training: 20 epochs · AdamW · Cosine LR

Hierarchical CNN (5-way) · 632,603 params · deep backbone, 5 expert heads

Input → [Conv][Conv][Conv] → FC(256) → Router (5-way)

```

├→ Linear(256→10) → digits    (0-9)
├→ Linear(256→9)  → letters   (A-J)
├→ Linear(256→15) → chinese    (CN-1...CN-15)
├→ Linear(256→10) → urdu      (alif...nun)
└→ Linear(256→10) → kannada   (KD-0...KD-9)

```

▲ Warm-up + joint training: P1 (3 ep) coarse router; P2 (22 ep) joint masked CE over all experts. Router accuracy 99.82% – all 5 scripts remain cleanly separated. Only 1 of 5 expert heads is active per inference pass (~20% dormant params).

Flat MLP · 671,542 params · no convolution, pure linear transforms (54 classes)

Input → Flatten(1024) → Linear(1024→512)+BN → Linear(512→256)+BN → Linear(256→54)

▲ All parameters always active. No conv layers – operates directly on raw pixel values.
Training: 20 epochs · AdamW · Cosine LR

5 Improvement Attempts

During development, we tried several modifications to close the gap between the hierarchical model and the flat CNN. Not all of them were kept in the final standardized setup, but each one clarified where the architecture helps and where extra complexity is not worth it.

Attempt	What Changed	What Happened	Takeaway
Joint training instead of a hard phase split	Use a short router warm-up, then optimize coarse loss and fine expert loss together.	This was the most consistently helpful change. It closed most of the hierarchical model's original accuracy gap.	The shared backbone learns better features when it receives coarse and fine supervision at the same time.
Top-2 / soft routing	Let the router keep more than one expert candidate instead of committing to a single branch.	It reduced the penalty of some routing mistakes, but the overall gain was small and inference became noticeably slower.	Useful as a robustness idea, but too expensive for the small improvement it delivered here.
Kannada augmentation and re-sampling	Increase Kannada exposure with stronger augmentation and more frequent sampling.	This helped a lot when Kannada was still a tiny domain, because the expert was otherwise under-trained.	Data balancing works when one expert is starved; after switching to official Kannada-MNIST, it was no longer the main bottleneck.

Attempt	What Changed	What Happened	Takeaway
Upgrade the expert from a linear head to a small expert network	Replace a single linear classifier with a lightweight expert-specific MLP block.	It produced only modest gains, and those gains were offset by extra latency and implementation complexity.	For this benchmark, better training and better data mattered more than making each expert head heavier.

Final choice: keep the hierarchical structure simple, use warm-up + joint training, and evaluate on the official Kannada-MNIST split rather than rely on a custom low-data Kannada subset.

We also tested earlier branching, where the network split into script-specific paths much sooner. That version fell to about **95%** accuracy, so we did not keep it. The likely reason is that handwritten scripts still share many low-level stroke patterns, including edges, curves, endpoints, and junctions. Unlike natural-image mixtures such as cats versus fish, the visual gap between scripts is not large enough to justify losing shared feature learning too early. In this project, specialization helped more at the decision stage than at the first representation stage.

6 Error Analysis

In hierarchical models, errors can be classified as **within-group** (router was correct, but fine expert was wrong) or **cross-group** (router itself misclassified the domain). The ratio reveals how well the routing mechanism isolates the classification problem.

Model	Total Errors	Within-Group	Cross-Group	Cross-Group %	Coarse Acc
Flat CNN	298	—	—	—	—
Hierarchical CNN (5-way)	332	274 (82.5%)	58	17.5%	99.82%
Flat MLP	2,442	—	—	—	—

Hierarchical CNN — Per-Expert Oracle Fine Accuracy

Expert	Classes	Oracle Fine Acc	Notes
Digits	10	99.58%	MNIST, strong visual separation
Letters	9	99.46%	EMNIST A–J (excl. I)
Chinese	15	99.14%	Stroke-heavy numerals — CNN captures patterns well
Urdu	10	99.36%	Cursive characters, UHaT dataset
Kannada	10	98.35%	Official Kannada-MNIST split

The hierarchical model still makes most of its mistakes within-group, while the router remains highly reliable overall (99.82% coarse accuracy). With the official Kannada-MNIST split, the Kannada expert reaches 98.35% oracle fine accuracy and no longer behaves like a fragile small-sample outlier.

7 Challenges & Solutions

CHALLENGE 1

Windows terminal encoding crash before training even started

► Problem

MLP scripts crashed with **UnicodeEncodeError (cp1252)** when printing Chinese characters to the Windows console before any training occurred.

► Solution

Added `sys.stdout.reconfigure(encoding='utf-8')` at the top of `main()` to force UTF-8 output regardless of the terminal's default code page.

► Problem

CHALLENGE 2

Chinese character recognition collapsed under MLP architectures

Flat MLP achieved only 25–80% per-class accuracy on Chinese characters. The models failed because MLP layers operate on flattened pixel vectors and cannot capture the local stroke patterns that distinguish characters like 二 from 三.

► Solution

Confirmed that **convolutional layers are non-negotiable for Chinese characters**. The flat CNN and hierarchical CNN both reach 97–100% per-class accuracy on Chinese numerals.

CHALLENGE 3

Joint training must compute masked loss per expert domain

► Problem

In the joint phase each expert shares the same minibatch stream, but only a subset of samples belongs to that expert's domain. A standard cross-entropy loss over the full batch would average in irrelevant rows and weaken the specialist gradient signal.

► Solution

Implemented a **masked cross-entropy** helper: for each batch, a boolean mask selects only the samples whose ground-truth domain matches the current expert, and loss is computed exclusively over those rows. This keeps warm-up + joint training stable while preserving expert specialization.

CHALLENGE 4

Adding a 5th script (Kannada) without creating a custom benchmark

► Problem

The earlier custom Kannada subset made the 5-script benchmark harder to defend: it used hand-selected classes, custom preprocessing, and a tiny sample count that was far out of scale with MNIST and EMNIST.

► Solution

Switched to the official **Kannada-MNIST** release (60,000 train / 10,000 test), which is already aligned with MNIST-style digit classification. This makes the Kannada branch standard, reproducible, and directly comparable to the rest of the project.

CHALLENGE 5

Pandas-before-torch DLL
crash on Windows
(pytorch_env)

► Problem

All three training scripts crashed with **OSError: Error loading c10.dll / shm.dll** when run with the conda GPU environment. Root cause: `import pandas as pd` appearing before `import torch` triggered DLL initialization conflicts in Windows.

► Solution

Reordered imports in all three scripts so `import torch; import torch.nn as nn` appears immediately after the standard library imports, before matplotlib, numpy, and pandas. Additionally, scripts must be invoked via `cmd /c "call activate.bat pytorch_env && python ..."` to properly set conda DLL search paths.

8 Key Findings

✓ Reliable routing keeps the hierarchy intact

The 5-way router reaches **99.82%** coarse accuracy on the official five-script benchmark. Only **58** of **32,466** test samples become cross-group errors, so script-level separation remains very strong.

✓ CNN is essential for complex scripts

CNN-based models reach 97–100% per-class accuracy on Chinese and Urdu characters. The Flat MLP falls to 25–80% on Chinese (strokes require spatial feature extraction) and 66–99% on Urdu (cursive forms). Digits and letters are learnable by MLP (97–99%) but Chinese/Urdu/Kannada are not.

△ Flat CNN still leads, but only narrowly

Flat CNN remains best at **99.08%**, while the Hierarchical CNN reaches **98.98%**. The remaining gap is only **0.10%**, showing that the modular hierarchy stays very competitive under the standardized Kannada-MNIST setup.

✗ CNNs still matter most on the visually complex scripts

Flat MLP improves slightly overall to **92.48%**, but it still struggles badly on Chinese and several Urdu/Kannada classes. Convolution remains the deciding factor for the difficult scripts, not just for raw parameter count.

★ **Architecture scales cleanly to new scripts**

Adding Kannada as a 5th script required only adding one Linear(256→10) expert head (+352 params) to the existing 4-way model. The backbone and all other expert heads were unchanged. This demonstrates the hierarchical design's **modularity** — new scripts plug in without disturbing existing ones.

— **Warm-up + joint training fits this model better**

A short router warm-up followed by **22 joint epochs** with masked cross-entropy gives the shared backbone useful fine-grained supervision from the start. Under the official Kannada-MNIST split, that recipe reaches **98.98%** without any custom Kannada-specific balancing tricks.

9 **Conclusion & Recommendations**

The hierarchical framework now shows that a tree-structured router + expert architecture can come very close to a flat monolithic CNN in accuracy (Hier CNN **98.98%** vs Flat CNN **99.08%**) while preserving clear engineering advantages: modular experts that can be independently retrained or replaced, ~20% dormant parameters during inference (4 of 5 expert heads idle per sample), and a lightweight warm-up + joint training recipe that is still easy to extend.

Extending from 4 to 5 scripts (adding Kannada) still required only adding one linear expert head, confirming that **the hierarchical architecture scales cleanly to new scripts**. With the official Kannada-MNIST split, the Kannada expert reaches **98.35%** oracle fine accuracy, and router accuracy remains **99.82%** across all five scripts.

Recommendation	Model	Why
Best accuracy	Flat CNN	99.08% — simplest to train, no routing overhead, single-pass inference
Best accuracy + modularity	Hier CNN	98.98% across 54 classes / 5 scripts — modular experts, 20% dormancy, and only a 0.10% gap to the flat CNN

Recommendation	Model	Why
Lowest memory footprint	Flat MLP	30 MB VRAM, 0.39 ms inference — but Chinese/Kannada accuracy is still much weaker than the CNN models